

Part C — Review of `api_tokens.py` ("Add API token support for /ask")

1. Every problem I found

Authentication — all of it reachable unauthenticated.

- SQL injection: the token is concatenated into the query — `Bearer ' OR '1'='1` logs in as row one.
- Tokens stored raw and logged in full; `row is None` is the only check, so revocation is absent.
- `auth.replace("Bearer ", "")` strips the word anywhere, not just a prefix; no header means "".
- 401 lacks detail/`WWW-Authenticate`; `auth` is a plain call, not a dependency, so it is skippable.

Tenancy and privacy

- `body.get("party_id", ...)` lets the caller name its own tenant — any token reads any party.
- `ac_code` comes from the body and is never checked against the token.
- `retrieve_chunks` gets no source-type filter, so private field reports are in scope.
- Question text is logged *and* stored in `usage_log`; the log also carries 80 chars of the answer.
- `usage_log` stores the token, not a token id, and `SELECT *` drags it into that log line.

Limits, cost and reliability

- `RATE` is an in-process dict: reset by every deploy, multiplied by workers, never evicted.
- The promised timer does not exist, so a paying token is 429 forever after five calls.
- It increments before the work, so failures burn quota; and there is no cap on `cost_inr` at all.
- No timeout on `call_llm`, and the DB session is held open across the whole 40-second call.
- `call_llm` and `db.execute` are sync inside `async def`, blocking the event loop, not one request.
- No `commit()`, a failed call writes no usage row, and the response drops citations and `cost_inr`.
- `body["question"]` raises `KeyError` → 500 not 400, and nothing caps its length.
- A new auth path arrived with no tests, and auth shares a file with the handler.

2. Top three by severity

1. **SQL injection in the auth path.** Anyone who reaches the endpoint authenticates as any customer and can read or drop the token table — on a public URL, that gets found.
2. **Caller-chosen party_id, unchecked ac_code, no source filter.** One campaign's credential reads another's private ground-worker reports — a DPDP breach we don't recover from.
3. **Raw tokens and question text in the logs and the database.** One log export hands over every live credential *and* what political operatives were asking.

3. The comment I'd post

Thanks for taking this on — the shape is right, and your two comments (open DB session, no LLM timeout) are the instincts I want. Three of these are blockers though, so I'd rather you rewrite than patch: the token is concatenated into the SQL string, so `Bearer ' OR '1'='1` logs in as someone else; `body.get("party_id", ...)` lets the caller pick its own tenant and `ac_code` isn't checked; and we store a hash, log a token id, and never persist question content — the last is a company commitment, not a preference.

It also needs a rate limit that survives a deploy, a cap on `cost_inr`, a timeout on `call_llm`, citations returned, and a test per gate. None of this is obvious the first time you write an auth path — grab me tomorrow and we'll pair on the token storage.